

¿Cómo funciona un programa de computadora?

Un programa hace que una computadora sea utilizable. Sin un programa, una computadora, incluso la más poderosa, no es más que un objeto. Del mismo modo, sin una pianista, un piano no es más que una caja de madera.

Las computadoras pueden realizar tareas muy complejas, pero esta habilidad no es innata. La naturaleza de una computadora es bastante diferente. Una computadora puede ejecutar solo operaciones extremadamente simples, por ejemplo, una computadora no puede comprender el valor de una función matemática complicada por sí misma, aunque no este más allá de los límites posibles en un futuro próximo.

Lenguajes naturales frente a lenguajes de programación

Un lenguaje es un medio (o una herramienta) para expresar y registrar pensamientos. Hay muchos lenguajes a nuestro alrededor. Algunos de ellos no requieren hablar ni escribir, como el lenguaje corporal. Es posible expresar tus sentimientos más profundos de manera muy precisa sin decir una sola palabra.

¿Qué compone a un lenguaje?

Podemos decir que cada lenguaje (maquina o natural, no importa) consta de los siguientes elementos:

- Un alfabeto: un conjunto de símbolos utilizados para formar palabras de un determinado lenguaje (por ejemplo, el alfabeto latino para el inglés, el alfabeto cirílico para el ruso, el Kanji para el japonés, y así sucesivamente).
- Un léxico: (también conocido como diccionario) un conjunto de palabras que el lenguaje ofrece a sus usuarios (por ejemplo, la palabra "computadora" proviene del diccionario en inglés, mientras que "computer" no, la palabra "chat").

¿Qué es Python?

Python es un lenguaje de programación de alto nivel, interpretado, orientado a objetos y de uso generalizado, con semántica dinámica, que se utiliza para la programación de propósito general.

¿Quién creó Python?

Una de las características sorprendentes de Python es el hecho de que en realidad es el trabajo de una persona. Por lo general, los grandes lenguajes de programación son desarrollados y publicados por grandes compañías que emplean a muchos profesionales, y debido a las normas de derechos de autor, es muy difícil nombrar a cualquiera de las personas involucradas en el proyecto. Python fue creado por Guido van Rossum, nacido en 1956 en Haarlem, Países Bajos.

¿Qué hace especial a Python?

- ¿Por qué los programadores, jóvenes y viejos, experimentados y novatos, quieren usarlo? ¿Cómo fue que las grandes empresas adoptaron Python e implementaron sus productos al usarlo? Existen muchas razones. Ya hemos enumerado algunas de ellas, pero vamos a enumerarlas de manera rápida.
- Es fácil de aprender: el tiempo necesario para aprender Python es más corto que en muchos otros lenguajes; esto significa que es posible comenzar la programación real más rápido.
 - Es fácil de enseñar: la carga de trabajo de enseñanza es menor que la que necesitan otros lenguajes; esto significa que el profesor puede poner más énfasis en las técnicas de programación generales.
 - Es fácil de usar: para escribir software nuevo, a menudo es posible escribir código más rápido cuando se emplea Python.
 - Es fácil de entender: a menudo, también es más fácil entender el código de otra persona más rápido si está escrito en Python.

Carlos Antonio Fitas Juárez

Modulo 2 Tipos de variables, datos, operaciones básicas de entrada y salida

La función `print()`

La palabra `print` es el nombre de una función. Eso no significa que donde quiera que aparezca esta palabra, será siempre el nombre de una función. El significado de la palabra proviene del contexto en el cual se haya utilizado la palabra.

Las funciones en python, sin embargo, son más flexibles y pueden contener más que sus parientes matemáticos.

Una función (en este contexto) es una parte separada del código de computadora el cual es capaz de:

- Causar algún efecto (por ejemplo, enviar texto a la terminal, crear un archivo, dibujar una imagen, reproducir un sonido etc); esto es algo completamente inaudito en el mundo de las matemáticas.
- Evaluar un valor (por ejemplo, la raíz cuadrada de un valor o la longitud de un texto dado) y devolverlo como el resultado de la función; esto es lo que hace que las funciones de python sean parientes de los conceptos matemáticos.
- De donde provienen las funciones?
- Pueden venir de python mismo. La función `print` es una de este tipo; dicha función es un valor asignado de python junto con su entorno (está integrada); no tienes que hacer nada especial (por ejemplo, pedirle a alguien algo) si quieres usarla.
- Pueden provenir de uno o varios de los módulos de python (llamados complementos); algunos de los módulos vienen con python, otros pueden requerir una instalación por separado, cual sea el caso, todos deben estar conectados explícitamente con el código (te mostraremos cómo hacer esto pronto).
- Puedes escribirlas tú mismo, colocando tantas funciones como desees y necesitas dentro de su programa para hacerlo más simple, claro y elegante.

El nombre de la función debe ser significativo (el nombre de la función `print` es evidente; imprime en la terminal).

Puntos clave

- 1 La función `print()` es una función integrada que imprime / envía un mensaje específico a la pantalla / ventanas de consola.
- 2 Las funciones integradas, al contrario de las funciones de librerías, por el contrario, están siempre disponibles, no tienen que ser importadas. Python 3.7.1 viene con 59 funciones incorporadas.
- 3 Para llamar a una función (invocación de función), debe utilizarse el nombre de la función, seguido de un paréntesis. Puedes pasar argumentos a una función colocando dentro de los paréntesis. Se deben separar los argumentos con comas, por ejemplo, `print("Hola", "Mundo!")`. Una función `print()` vacía `print()` imprimirá una línea vacía a la pantalla.
- 4 Las cadenas de Python están delimitadas por comillas, por ejemplo, "Soy una cadena", o 'Yo soy una cadena, también!'.
- 5 Los programas de computadora son colecciones de instrucciones, una instrucción es un comando para realizar una tarea específica cuando se ejecuta, por ejemplo, imprimir un determinado mensaje en la pantalla.
- 6 En las cadenas de Python, la barra diagonal (`\`) hace (`\n`) o (`\r`) es un carácter especial que indica que el siguiente carácter tiene un significado diferente, por ejemplo, `\n` es el carácter de nueva línea, comienza una nueva línea después.
- 7 Los argumentos posicionales son aquellos cuyo significado viene dictado por su posición, por ejemplo, el segundo argumento se envía después del primero, el tercero se envía después del segundo, etc.

Comparación Operador de igualdad Módulo 3 Tomando decisiones en Python

¿Son dos valores iguales?

Para hacer esta pregunta, se utiliza el `==` operador (igual a).

- `=` es un operador de asignación, por ejemplo `a = b` asigna a la variable `a` el valor de `b`.

- `==` es una pregunta (¿Son estos valores iguales?). Así que `a == b` compara `a` y `b`.

Es un operador binario (conlazado del lado izquierdo). Necesita dos argumentos y verificar si son iguales.

Ejemplos: `2 == 2` `True`, `1 == 2` `False`.

Operadores de comparación: mayor o igual que.

El operador mayor que tiene otra variante especial, una variante no estricta, pero se denota de manera diferente que la notación aritmética clásica, `>=` (mayor o igual que). `>` da signos subyacentes, no uno.

Ambos operadores (estricto y no estricto), así como los otros dos que se analizan en la siguiente sección, son operadores binarios (conlazado del lado izquierdo). Su prioridad es mayor que la mostrada por `==` y `!=`.

Si queremos saber si tenemos que usar un guro o no, nos hacemos la siguiente pregunta: `centigrados - outside >= 0.0` # mayor o igual que.

Operadores de comparación: menor o igual que.

Como probablemente ya hayas adivinado, los operadores utilizados en este caso son: El operador `<` (menor que) y su hermano no estricto `<=` (menor o igual que).

Ejemplos: `current_veto_mph < 85` # menor que

`current_vet_mph <= 85` # menor o igual que

Existen al menos dos posibilidades: primero puedes memorizar (al menos en una variable) y utilizarlo más tarde (¿cómo haces eso? Bueno, utilizando una variable arbitraria).

`answer = number_of_lions >= number_of_lionesses`

Punto clave

Los operadores de comparación (o también denominados operadores relacionales) se utilizan para comparar valores. La siguiente tabla ilustra cómo funcionan los operadores de comparación asumiendo que $x=0$, $y=7$ y $z=0$.

Operador	Descripción	Ejemplo
$=$	Devuelve si los valores de los operandos son iguales, y False de lo contrario	$x = y$ # False $x = z$ # True
$!=$	Devuelve True si los valores de los operandos no son iguales, y False de lo contrario	$x != y$ # True $x != z$ # False
$>$	Devuelve True si el valor del operando izquierdo es mayor que el valor del operando derecho, y False de lo contrario	$x > y$ # False $y > z$ # True
$<$	Devuelve True si el valor del operando izquierdo es menor que el valor del operando derecho, y False de lo contrario	$x < y$ # True $x < z$ # False
$>=$	Devuelve True si el valor del operando izquierdo es mayor o igual al valor del operando derecho, y False de lo contrario	$x >= y$ # False $x >= z$ # True $y >= z$ # True
$<=$	Devuelve True si el valor del operando izquierdo es menor o igual al valor del operando derecho, y False de lo contrario	$x <= y$ # True $x <= z$ # True $y <= z$ # False

Una sentencia `if-else`, por ejemplo:

`x = 10`

`if x < 10: # Condición`

`print("x es menor que 10")`

`else:`

`print("x es mayor o igual a 10") # Verdadero`

Ejemplo 2

`x = 10`

`if x >= 5: # True`

`print("x >= 5")`

`if x > 0: # True`

`print("x > 0")`

`if x > 10: # False`

`print("x > 10")`

`else:`

`print("se ejecutará el else")`

Modulo 4

Funciones, Tuplas, Dictionaries, Exceptions, y Pruebas de código

¿Por qué necesitamos funciones?

Hasta ahora ha

número # : 01 < x 7

("01 sup version de x") - 1009

Descomposición

Es más común que un programa sea tan largo y complejo que no puede ser asignado a un solo desarrollador, y en su lugar un equipo de desarrolladores trabajaran en él. El problema, debe ser dividido entre varios desarrolladores de una manera en que se pueda asegurar su eficiencia y cooperación.

¿De dónde provienen las funciones?

En general, las funciones provienen de al menos tres lugares:

- De Python mismo: varias funciones (como `print()`) son una parte integral de Python, y siempre están disponibles sin algún esfuerzo adicional del programador; se les llama a estas funciones integradas.
- De los módulos preinstalados de Python: muchas de las funciones, las cuales comúnmente son menos utilizadas que las integradas, están disponibles en módulos instalados juntamente con Python; para poder utilizar estas funciones el programador debe realizar algunos pasos adicionales (se explicaran cerca de esto en un momento).
- Directamente del código: tu puedes escribir tus propias funciones, ubicarlas dentro del código, y usarlas libremente.
- Existe una posibilidad más, pero se relaciona con clases se omitirá por ahora.

Una función es un bloque de código que realiza una tarea específica cuando la función es llamada (invocada). Las funciones son útiles para hacer que el código sea reutilizable, que este mejor organizado y mas legible. Las funciones contienen parámetros y pueden regresar valores.

Funciones definidas por el usuario las cuales son partes importantes de Python (como lo es la función `print()`).

Funciones parametrizadas

El potencial completo de una función se revela cuando puede ser equipada con una interfaz que es capaz de aceptar provenientes de la invocación. Dichos datos pueden modificar el comportamiento de la función, haciéndola más flexible > adaptable a condiciones cambiantes

Un parámetro es una variable, pero existen dos factores que hacen a un parámetro diferente:

- Los parámetros solo existen dentro de las funciones en donde han sido definidos y el único lugar donde un parámetro puede ser definido es entre las parentesis después del nombre de la función, donde se encuentra la palabra reservada `def`
- La asignación de un valor a un parámetro de una función se hace en el momento que la función se manda llamar o se invoca especificando el argumento correspondiente.

`def function(parameters) ## ##`

Recuerda que:

- Los parámetros solo existen dentro de las funciones (este es su entorno natural)
- Los argumentos existen fuera de las funciones, y son los que pasan los valores a los parámetros correspondientes.

Antes claves Tuplas

Las Tuplas son colecciones de datos ordenadas e inmutables. Se puede pensar en ellas como listas inmutables. Se definen con parentesis:

```
mz_tuple = (1, 2, True, "una cadena", (3, 4), [5, 6], None)
```

```
print(mz_tuple)
```

```
mz_list = [1, 2, True, "una cadena", (3, 4), [5, 6], None]
```

```
print(mz_list)
```